

MODULE 3.1

Cloud Platform Architecture over Virtualized Data Centers

THE ULTIMATE "I HAVE 3 HOURS" CRASH COURSE

Full Stack Dev Edition — With Dev Analogies & Memory Tricks

Reference: Kai Hwang, Geoffrey C. Fox, Jack Dongarra

1. DATA CENTER DESIGN — WAREHOUSE-SCALE DATA CENTERS (WSC)

▶ 1.1 What Even IS a Data Center?

A data center is basically a MASSIVE building stuffed with servers, all connected by a huge network. Dennis Gannon said it best: 'The cloud is built on massive data centers.'

Two main types:

- **Warehouse-Scale DC (WSC):** Shopping-mall sized. ~11x a football field. Houses 400,000 to 1 MILLION servers. All under one roof.
- **Modular DC (MDC):** Fits in a 40-foot shipping container. Portable. ~1,000 servers. Can be trucked anywhere.

 **DEV ANALOGY:** You deploy to AWS/GCP/Azure? Your code runs on a physical server inside one of these giant buildings. EC2 instance = literally a server in a warehouse somewhere.

 **MEMORY TRICK:** WSC = 'Walmart-Scale Computing'

Think Walmart Supercenter = WSC (huge, one roof). MDC = Food Truck (small, portable, goes where needed).

▶ 1.2 Economics of Scale — Why Bigger = Cheaper

The entire reason companies build these monster facilities is MONEY. Bigger = lower cost per server.

Monthly costs for a LARGE data center (400,000+ servers):

- **Network cost:** \$13/Mbps
- **Storage cost:** \$0.40/GB

Now compare a SMALL data center (1,000 servers):

- Network cost is 7x MORE expensive per unit
- Storage cost is 5.7x MORE expensive per unit

 **DEV ANALOGY:** Same reason companies use monorepos or shared CI/CD — shared infra costs less per unit. Buying 1M hard drives = massive bulk discount vs 1,000.

MEMORY TRICK: '7-Up and 5.7 Down'

Network = 7x more expensive small vs large. Storage = 5.7x. The network number is bigger because networking gear is pricier at small scale.

 **EXAM TIP:** They LOVE asking cost ratios. Network = 7x, Storage = 5.7x. Classic number-recall question.

► 1.3 Data Center Construction — The Hardware

Built with commercially available, off-the-shelf components. No custom silicon.

What's inside a single server:

- Processor sockets with multicore CPUs
- Cache hierarchies (L1, L2, L3)
- Local shared DRAM (main memory)
- Directly attached disk drives

Example Config — 2,000 server DC (MEMORIZE THIS):

- Each server: 8 GB DRAM + four 1 TB disk drives
- 40 servers per rack, each connected via 1 Gbps link to a rack-level switch
- Each rack switch: 8 extra 1 Gbps uplink ports → cluster-level switch

Network hierarchy:

- Server → Rack Switch (via 1 Gbps)
- Rack Switch → Cluster Switch (via 8 × 1 Gbps uplinks)
- All racks accessible through cluster-level switch

 **DEV ANALOGY:** Individual servers = app instances. Rack switch = load balancer for one service. Cluster switch = API gateway routing between all services.

MEMORY TRICK: '40-8-1' Rule

40 servers per rack, 8 uplinks per rack switch, 1 Gbps per link. Like a sports jersey: 4-8-1.

► 1.4 Cooling System — Because Servers Run HOT

Cooling is one of THE most critical and expensive parts of running a DC.

Raised Floor Design

- Raised floors — steel grid on stanchions 2-4 feet above concrete floor
- Gap underneath hides cables, power lines, and distributes cool air to racks
- It's an under-floor highway for cold air

CRAC Unit (Computer Room Air Conditioning)

- CRAC units blow cold air INTO the raised floor plenum (under-floor space), pressurizing it
- Cold air escapes through perforated tiles placed in front of server racks
- Hot air from servers → back to CRAC intakes → cooled → pushed into plenum again
- Incoming coolant temperature: 12-14°C. Warm coolant returns to a chiller.

MEMORY TRICK: CRAC = 'Cold Room AC'

Pushes cold air through the floor like an upside-down AC. Standing on a subway grate with cool air = perforated tiles. Coolant: 12-14°C (a dozen to fourteen).

Hot Aisle / Cold Aisle Arrangement

- Racks in alternating rows: COLD aisle → HOT aisle → COLD aisle → HOT aisle
- Cold aisles face perforated tiles (cool air intake)
- Hot aisles face rear of servers (exhaust side) → heat returns to CRAC
- Purpose: PREVENTS mixing hot and cold air = no wasted energy, no hot spots

 **DEV ANALOGY:** Like keeping Redux state immutable — don't let hot data (changes) contaminate cold data (current state). Hot/cold separation = physical separation of concerns.

 **EXAM TIP:** 'Why hot/cold aisle?' → Avoid hot spots, save cooling energy. Which faces perforated tiles? → Cold aisle. Hot aisle = server rears (exhaust).

MEMORY TRICK: 'CRAP' cooling flow

CRAC → Raised floor → Aisle (cold) → through servers → Produces hot air → back to CRAC. Loop!

2. DATA CENTER INTERCONNECTION NETWORKS

The interconnection network is the nervous system of the DC. Connects ALL servers. If the network sucks, everything sucks.

► 2.1 Five Core Requirements

#	Requirement	What It Means
1	Low Latency	Minimal delay between nodes. Like ping time — lower is better.
2	High Bandwidth	Fat pipes. Transfer tons of data quickly.
3	Low Cost	Cheap to deploy and maintain.
4	MPI Support	Message Passing Interface — standard for distributed/parallel computing.
5	Fault Tolerance	Keep working when components fail. No single point of failure.

 **MEMORY TRICK:** 'Lazy Boys Love Cheap Motorcycles Fast'

Low latency, Bandwidth, Low Cost, MPI, Fault tolerance = L-B-L-M-F.

► 2.2 Design Considerations (5 Sub-Topics)

(a) Application Traffic Support

- Must support ALL MPI patterns: point-to-point AND collective
- **Need** high bisection bandwidth (BW across the middle split of network)
- One-to-many comms for distributed file access (metadata master → slave nodes)
- **MapReduce support:** must perform map and reduce at high speed
- Network must handle various traffic patterns from different cloud apps

 **DEV ANALOGY:** Like your message broker (Kafka/RabbitMQ): needs pub/sub (one-to-many), req/response (point-to-point), fan-out (MapReduce). If broker fails, microservices fall apart.

 **EXAM TIP:** Bisection Bandwidth = BW if you cut the network in half. High bisection BW = good for all-to-all comms. They love this definition.

(b) Network Expandability

- Network must be expandable as servers are added
- Fat-tree and crossbar networks use low-cost Ethernet switches
- Expansion gets hard when server count jumps sharply

 **DEV ANALOGY:** Like horizontal scaling in K8s. Add more pods/nodes and network figures it out. Fat-tree = well-designed K8s networking plugin.

(c) Fault Tolerance

- Tolerate link/switch failures via MULTIPLE PATHS between any two servers
- Server fault tolerance: replicate data + computation across redundant servers
- Both software AND hardware redundancy needed
- Software layer is AWARE of failures; packet forwarding avoids broken links
- Drivers handle failures transparently — cloud operations unaffected

 **DEV ANALOGY:** Multiple K8s replicas + health checks. If one pod dies, traffic reroutes. Service mesh (Istio) = software layer 'aware' of failures.

(d) Graceful Degradation

- System should NOT collapse when nodes fail — just slow down
- **Hot-swappable** components: replace broken parts WITHOUT shutting down
- No critical paths or single points of failure

 **MEMORY TRICK:** Graceful Degradation = 'Netflix Mode'

When overloaded, Netflix lowers video quality instead of crashing. Work worse, not stop working. Hot-swappable = USB — plug in/out while running.

(e) Switch-Centric vs. Server-Centric Design

Aspect	Switch-Centric	Server-Centric
Who routes?	Switches do all routing	Servers participate in routing too
OS changes?	NO changes to server OS	YES — modifies OS, special drivers
Separation	Clean: network vs compute	Blurred: servers = also network nodes

Example

Fat-Tree (used in WSC)

BCube (used in MDC)

 **DEV ANALOGY:** Switch-centric = traditional Nginx reverse proxy doing all routing. Server-centric = service mesh where each server has a sidecar proxy (Envoy) and participates in routing.

 **MEMORY TRICK:** 'Fat-Tree = Switches, BCube = Servers'

Fat-Tree = Switch-centric (for WSC). BCube = Server-centric (for MDC containers). B-Cube = B-ox (container).

3. FAT-TREE INTERCONNECTION TOPOLOGY (for WSC)

Fat-tree is THE go-to network design for warehouse-scale data centers.

 **EXAM TIP:** CHECK YOUR PDF PAGE 3-4 for the fat-tree diagram. Be able to draw it from memory!

▶ 3.1 Fat-Tree Structure

Hierarchical: bottom layer = server nodes, top layers = switches.

Switch Type	Position	Role
Edge Switches	Bottom of switch layers	Directly connect to server nodes
Aggregation Switches	Middle layer	Aggregate multiple edge switches
Core Switches	Top layer	Connect different pods together

POD = A group of aggregation switches + edge switches + their server nodes. A 'neighborhood' in the DC.

 **DEV ANALOGY:** Edge = Load Balancers (closest to servers). Aggregation = API Gateway. Core = Global LB (connects regions/pods). POD = one K8s namespace.

 **MEMORY TRICK:** 'EAC-P' = Fat-Tree components

'Everyone Always Craves Pizza' = Edge → Aggregation → Core, organized into Pods.

▶ 3.2 Fault Tolerance in Fat-Tree

- Multiple paths between ANY two server nodes
- Link fails → alternate path used automatically
- Aggregation switch dies → rest of network is FINE
- Edge switch dies → only a SMALL number of servers directly below are affected
- Extra switches in each pod = extra bandwidth

 **EXAM TIP:** 'What if an aggregation switch fails?' → Does NOT affect whole network connectivity. Edge switch failure → only its directly-connected servers affected.

Why fat-tree wins for WSC:

- Redundancy (multiple paths)
- Scalability with LOW-COST Ethernet switches
- Multiple-path fault tolerance
- High bandwidth via extra switches



MEMORY TRICK: 'Fat-Tree = Fat Wallet'

Rich features (redundancy, scale, fault tolerance) using CHEAP Ethernet switches. Not expensive proprietary gear.

4. MODULAR DATA CENTERS (MDC) — Shipping Containers

Imagine stuffing a data center into a shipping container. That's literally what this is.

► 4.1 Overview

- Modern DC can look like a shipping yard — server clusters in truck-towed containers
- **SGI ICE Cube** = the famous MDC example (REMEMBER THIS NAME)
- MDC can be placed ANYWHERE capacity is needed
- Available in 40-foot portable and multi-story configs

 **DEV ANALOGY:** WSC = monolithic bare-metal deployment. MDC = Docker containers — self-contained, portable, scale by adding more. The shipping container IS a physical Docker container for servers.

► 4.2 Internal Structure

- Hundreds of blade servers in racks along container walls
- Fans push heated air through a heat exchanger → cools → circulates back → continuous loop

Rack Server vs. Blade Server (EXAM FAVORITE):

Feature	Rack Server	Blade Server
Form Factor	Standard box mounted in rack	Thin modular card in shared chassis
Components	Full: CPU, RAM, storage, PSU, NIC	Only CPUs, memory, NIC. Power/IO shared from chassis
Mounting	Screws or rails	Slides into chassis slot
Purpose	General-purpose, wide configs	High density, for MDC/containers
Think of it as...	Desktop PC — all in one box	Laptop/tablet — thin, shares resources

 **MEMORY TRICK:** 'Blade = Slim, Rack = Regular'

Blade = thin like a blade, slides in, shares power/IO from chassis. Rack = full independent box, screws/rails.

► 4.3 Capacity — SGI ICE Cube

- **46,080** processing cores per container, OR
- **30 PB** of storage per container
- Multiple containers = massive DC that looks like a shipping yard

 **MEMORY TRICK: '46K cores, 30PB storage'**

~46 thousand cores (container is ~40 foot, so 46 is close). 30PB = thirty petabytes.

► 4.4 WHY Use Container-Based DCs?

- Lower power consumption
- Higher computer density (more compute per sq ft)
- Mobility: move to where electricity is CHEAPER
- Better cooling water supplies (rivers, cold climates)
- Cheaper housing for maintenance engineers
- High mobility — not fixed to one location unlike WSC

 **DEV ANALOGY: Same reasons for Docker + K8s: flexibility, portability, cost optimization, scale by adding units. MDCs = Docker for hardware.**

► 4.5 Construction Process

Stage	What	Details
1	Single server system	Start with one — the atomic unit
2	Rack system (40 servers)	Takes about half a day to build
3	Full container (~1000 servers)	Multiple racks, power, networking, cooling, full testing

- Full MDC construction + testing: a FEW DAYS (with all components ready)
- Container is truck-towable, weatherproof, easy to transport
- Cooling efficiency: vary water and airflow; handle seasonal loads

 **MEMORY TRICK: '1 → 40 → 1000' stages**

1 server → 40 (rack, half-day) → 1000 (container, few days). Like Legos: one brick, then wall, then whole set.

► 4.6 Portability and Applications

- Healthcare: Install DCs at clinic sites (data stays local)
- Edge computing: Put compute CLOSE to end users

 **DEV ANALOGY:** Like deploying Cloudflare Workers or Vercel Edge — but for PHYSICAL infrastructure. Truck a mini DC to each hospital, factory, or remote site.

5. WSC vs. MDC — THE ULTIMATE COMPARISON

This table is exam gold. Know every row.

Aspect	WSC (Warehouse-Scale)	MDC (Modular)
Definition	Large centralized facility, 100K+ servers, one massive system	Pre-built containers: servers, cooling, power, networking
Design	Warehouse facility, entire infra planned upfront	Container-based; add/remove as demand grows
Scalability	Massive from the start	Incremental — add more containers
Deployment	LONGER — build whole facility first	FASTER — pre-built, just install + connect
Typical Users	Google, Amazon, Microsoft	Edge computing, flexible orgs, rapid deploy
Management	Entire DC = one big computer	Each module = self-contained unit
Network	Fat-Tree (switch-centric)	BCube (server-centric) + MDCube between



MEMORY TRICK: 'WSC = Walmart, MDC = McDonald's'

WSC = one massive Walmart Supercenter. MDC = McDonald's franchises — standardized, deploy anywhere, add as needed.

6. BCube NETWORK — Server-Centric for MDC

BCube is the networking solution for modular data centers. Servers help with routing (not just switches).

Reference: C. Guo et al., ACM SIGCOMM, Oct. 2009

 **EXAM TIP:** CHECK PDF PAGE 6 for BCube diagrams. Circles = Servers, Rectangles = Switches.

► 6.1 BCube Architecture

Built in layers (hierarchical):

- **BCube0 (Level 0):** The bottom layer. All server nodes. Level 1 switches on top.
- **BCube1 (Level 1):** Multiple BCube0 units connected by another layer of switches.
- **BCube2, BCube3...:** Keep stacking! Each higher level connects multiple units from below.

 **DEV ANALOGY:** Like React component composition: **BCube0 = atomic Button**. **BCube1 = Form (multiple Buttons)**. **BCube2 = Page (multiple Forms)**. Each level wraps the level below.

► 6.2 Key Features of BCube

- **Multiple Paths:** Multiple routes between any two nodes → extra bandwidth + fault tolerance
- **OS Kernel Module:** Installs a kernel module in server OS to perform routing. THIS makes it server-centric.
- **Cloud App Support:** Multiple paths = extra bandwidth for cloud communication patterns
- **Building Block Design:** BCube is a building block — used INSIDE each server container

 **MEMORY TRICK:** 'BCube = Brain in every Cube'

The 'B' = Brain. Every server has a brain (kernel module) for routing. In fat-tree only switches think. In BCube every server routes packets.

► 6.3 MDCube — Connecting Multiple Containers

BCube = INSIDE one container. MDCube = BETWEEN containers.

- **BCube** = intra-container networking (inside one container)
- **MDCube** (Modularized Datacenter Cube) = inter-container networking (between containers)
- MDCube connects multiple BCube containers using high-speed switches
- **Example:** 2-D MDCube = 9 BCube containers in a 3×3 grid

 **DEV ANALOGY:** BCube = networking inside a single Docker container (localhost). MDCube = Docker bridge network connecting multiple containers. BCube = intra-service, MDCube = inter-service.

 **MEMORY TRICK:** 'B = Box (inside), MD = Many Docks (between)'

BCube = inside one Box. MDCube = Many Docking stations. 2-D MDCube = 9 = 3×3 tic-tac-toe board.

 **EXAM TIP:** 'Difference between BCube and MDCube?' → BCube = inside (intra), MDCube = between (inter). BCube is server-centric. 2-D MDCube = 9 BCube containers.

7. DATA CENTER MANAGEMENT ISSUES (8 Issues)

After building the DC, you manage it. All 8 are testable.

#	Issue	What It Means (Dev-Friendly)
1	Make Users Happy	Quality service for 30+ YEARS. Long-term SLA.
2	Controlled Info Flow	Streamlined data flow. Goal = sustained services + HA.
3	Multi-user Mgmt	Traffic, DB updates, server maintenance — simultaneously for many users.
4	Scalability for DB Growth	Storage, processing, I/O, power, cooling — each independently scalable.
5	Reliability (Virtualized)	Failover, fault tolerance, VM live migration for disaster recovery.
6	Low Cost	Minimize ALL costs: hardware, power, networking, administration.
7	Security & Data Protection	Privacy, defense vs network attacks, data integrity.
8	Green IT	Save power, renewable energy, optimized cooling, energy-proportional computing.

 **DEV ANALOGY:** Maps to DevOps/SRE: 1=SLA, 2=CI/CD, 3=multi-tenancy, 4=auto-scaling, 5=K8s self-healing, 6=FinOps, 7=DevSecOps, 8=carbon-aware computing.

 **MEMORY TRICK:** 'Happy CaMS Really Love Saving Green'

Happy users, Controlled flow, Multi-user, Scalability, Reliability, Low cost, Security, Green IT.

Key terms within management:

- **VM Live Migration:** Moving a RUNNING VM between physical servers with ZERO downtime. Critical for load balancing + fault tolerance.
- **High Availability (HA):** Services stay accessible with minimal downtime. THE #1 goal.

- **Energy-Proportional Computing:** Servers use power proportional to workload. Idle = near-zero power.

 **DEV ANALOGY:** VM Live Migration = zero-downtime deployment. Deploy new version while old runs, then switch traffic. Same concept — move running workload, users notice nothing.

8. SPEED ROUND — ALL KEY TERMS

Rapid-fire definitions. Scan 10 minutes before exam.

Term	Definition (Exam-Ready)
WSC	Warehouse-Scale DC. 400K-1M servers. One massive system. Google/Amazon/Microsoft.
MDC	Modular DC. Portable, container-based. Deploy anywhere. Scale incrementally.
Economics of Scale	Bigger DC = cheaper/unit. Network: 7x cheaper. Storage: 5.7x cheaper.
CRAC Unit	Computer Room AC. Cold air → raised floor → perforated tiles → cools racks.
Hot/Cold Aisle	Alternating aisles: cold (intake) / hot (exhaust). Prevents mixing. Saves energy.
Fat-Tree	Switch-centric for WSC. Edge→Agg→Core + Pods. Multiple paths = fault tolerant.
BCube	Server-centric for MDC. Kernel module routes. Layered: BCube0→BCube1→...
MDCube	Inter-container network. Connects BCube containers. 2-D = 9 containers (3×3).
MPI	Message Passing Interface. Parallel computing comms. Point-to-point + collective.
Bisection BW	BW across middle split. High = good for all-to-all communication.
Graceful Degradation	System slows but doesn't CRASH when components fail.
Hot-Swappable	Replace components WITHOUT shutting down. Like USB plug-and-play.
VM Live Migration	Move running VM between servers. Zero downtime. Load balancing + FT.
High Availability	Services stay accessible, minimal downtime. Primary DC goal.
Blade Server	Thin card in shared chassis. CPU+RAM only. Shares power/IO. High density.
Rack Server	Standard box in rack. Self-contained. General purpose. Screws/rails.

FINAL CHEAT SHEET — ALL MNEMONICS

Topic	Memory Trick
WSC vs MDC	WSC = Walmart (huge, one roof). MDC = McDonald's (franchise, anywhere).
Cost ratios	7 (network) and 5.7 (storage) — small DC is this much pricier.
Server config	40-8-1: 40 servers/rack, 8 uplinks, 1 Gbps each.
Cooling flow	CRAC → Raised floor → Cold aisle → Servers → Hot aisle → CRAC. 12-14°C.
5 Network Reqs	'Lazy Boys Love Cheap Motorcycles Fast' = Low Lat, BW, Low Cost, MPI, FT.
Fat-Tree switches	'Everyone Always Craves Pizza' = Edge → Agg → Core, in Pods.
Fat-Tree = WSC	Switch-centric. Cheap Ethernet. If agg switch dies = network OK.
BCube = MDC	Server-centric. Kernel module. Inside container. MDCube = between.
MDCube	2-D MDCube = 9 BCube containers (3×3 tic-tac-toe).
SGI ICE Cube	46,080 cores OR 30 PB storage per container.
MDC construction	1 → 40 (half-day) → 1000 (few days).
8 Mgmt Issues	'Happy CaMS Really Love Saving Green'.
Blade vs Rack	Blade = thin, slides in, shares power. Rack = full box, self-contained.

You've got this. Go crush that exam! 💧